ChainDocs — Децентрализованная система документооборота

Доклад о системе | Версия 1.2.0 | 2026-02-22

Введение

Проблема современного документооборота

В современном мире организации сталкиваются с критическими проблемами при управлении документами:

Проблемы:

- **Неизменяемость** — документы могут быть изменены постфактум
- **Подтверждение авторства** — сложно доказать кто создал документ
- **Аудит изменений** — отсутствие полной истории операций
- **Доверие между сторонами** — необходимость в доверенном посреднике
- **Централизация** — единая точка отказа и контроля

Решение — ChainDocs

ChainDocs — это децентрализованная блокчейн-система для документооборота с криптографической подписью и P2P коммуникацией между узлами.

Ключевые преимущества:

- Неизменяемое хранение хэшей документов в блокчейне
- Криптографическая подпись документов (Ed25519)
- Мульти-подписи блоков (консенсус 51%+)
- P2P сеть между клиентами для децентрализации
- Динамический расчёт консенсуса от активных ключей
- Категории документов для организации
- Веб-интерфейс с безопасной аутентификацией

Архитектура системы

Компоненты

```
┌──────────────────────────────────────────────────────────┐
│              ChainDocs Server (Go)                       │
│  ┌──────────┐  ┌──────────┐  ┌──────────────────┐      │
│  │ REST API │  │ Blockchain│  │   Web Interface  │      │
│  │ (chi/v5) │  │  (bbolt) │  │  (HTMX/Alpine)   │      │
│  └──────────┘  └──────────┘  └──────────────────┘      │
└──────────────────────────────────────────────────────────┘
              ▲                  ▲
              │                  │
     ┌────────┴─────────┐  ┌─────┴────────────┐
     │   Client 1       │◄─┤   Client N       │
     │  (Daemon + P2P)  │  │  (Daemon + P2P)  │
     └──────────────────┘  └──────────────────┘
              ▲                  ▲
              └──────────┬───────┘
                         │
                    P2P Mesh
              (WebSocket connections)
```

Модули

internal/block

Структура блока и операции:

- `Block` — основная структура с хэшем документа
- `Signature` — подпись блока клиентом
- `DocumentSignature` — подпись документа владельцем
- `NewBlock()` — создание нового блока
- `CalculateHash()` — вычисление SHA-256 хэша
- `Verify()` — проверка целостности блока
- `ConsensusReached()` — проверка достижения консенсуса

internal/crypto

Криптографические операции:
- **Ed25519** — цифровые подписи блоков и документов
- **AES-256-GCM** — шифрование приватных ключей
- **scrypt** — KDF для получения ключа из пароля (N=32768)
- **SHA-256** — хэширование документов

internal/storage

Хранилище данных на основе bbolt:
- buckets: blocks, height, pubkeys, activity, categories, revoked
- ACID транзакции
- Встроенная индексация по высоте и хэшу

internal/p2p

P2P коммуникация между клиентами:
- WebSocket соединения
- 4 типа сообщений (block_announce, block_request, block_response, peer_list)
- Обнаружение новых пиров
- Поддержание соединений

pkg/logger

Структурированное логирование:
- Уровни: debug, info, warn, error
- Форматы: text, json
- Ротация через lumberjack (max 100MB, 5 файлов)

pkg/metrics

Prometheus метрики:
- `chaindocs_blocks_total` — всего блоков
- `chaindocs_active_keys` — активных ключей (24ч)
- `chaindocs_consensus_percent` — процент консенсуса
- `chaindocs_request_duration_seconds` — время запросов

P2P Архитектура

Зачем нужна P2P сеть?

Проблема централизованной системы:
- Единая точка отказа (сервер)
- Сервер должен быть всегда онлайн
- Клиенты не могут обмениваться данными напрямую
- При падении сервера система перестаёт работать

Решение через P2P:
- Клиенты соединяются напрямую друг с другом

- Данные реплицируются между узлами
- Отказ одного узла не влияет на систему
- Децентрализация и отказоустойчивость

Как работает P2P в ChainDocs

Структура сообщения

```go
type Message struct {
    Type      MessageType  // тип сообщения
    PeerID    string       // ID отправителя
    Block     *block.Block // блок (если есть)
    BlockHash string       // хэш блока
    Peers     []string     // список пиров
    Timestamp string       // время
}
```

Типы сообщений

block_announce — уведомление о новом блоке:

```json
{
  "type": "block_announce",
  "peer_id": "client1",
  "block_hash": "abc123...",
  "timestamp": "2026-02-22T12:00:00Z"
}
```

block_request — запрос блока:

```json
{
  "type": "block_request",
  "peer_id": "client2",
  "block_hash": "abc123...",
  "timestamp": "2026-02-22T12:00:01Z"
}
```

block_response — ответ с блоком:

```json
{
  "type": "block_response",
  "peer_id": "client1",
  "block": {...},
  "timestamp": "2026-02-22T12:00:02Z"
}
```

peer_list — обмен списками пиров:

```json
{
  "type": "peer_list",
  "peer_id": "client1",
  "peers": ["ws://server1:8081/p2p", "ws://server2:8081/p2p"],
  "timestamp": "2026-02-22T12:00:00Z"
}
```

Процесс подключения

```
1. Клиент запускается с списком бутстрап пиров
2. Подключается к известным адресам через WebSocket
3. Отправляет приветственное сообщение (peer_list)
4. Получает список других пиров
```

```
5. Подключается к новым пирам
6. Поддерживает соединение (heartbeat каждые 30с)
```

Обнаружение пиров (Discovery)

```
Client A          Client B          Client C
   |                 |                 |
   |──peer_list───>|                 |
   |                 |──peer_list───>|
   |                 |                 |
   |<──────────────┼──peer_list──────|
   |                 |                 |
   |──connect──────>|                 |
   |<──────────────|                 |
   |                 |                 |
```

Синхронизация блоков

```
1. Клиент получает новый блок от сервера
2. Подписывает блок своим ключом
3. Транслирует блок всем подключенным пирам
4. Пиры получают блок и тоже подписывают
5. Консенсус достигается быстрее благодаря P2P
```

Преимущества P2P архитектуры

Отказоустойчивость:

- При падении одного клиента другие продолжают работать
- Данные реплицируются между узлами
- Нет единой точки отказа

Масштабируемость:

- Каждый новый клиент улучшает сеть
- Больше пиров = больше путей для данных
- Нагрузка распределяется между узлами

Безопасность:

- Данные проверяются криптографически
- Невозможно подделать блок без ключа
- P2P сообщения подписываются

Децентрализация:

- Нет центрального контроллера
- Все узлы равны
- Решения принимаются консенсусом

Безопасность

Аутентификация в веб-интерфейсе

- Токен через переменную окружения `CHAINDOCS_AUTH_TOKEN`
- Генерация случайного токена при старте
- Страница входа `/web/login`
- Bearer токен в `Authorization` header
- API endpoints без аутентификации (для интеграции)

Криптография

Подписи:

- Ed25519 для подписей блоков и документов
- Детерминированные подписи (одни данные = одна подпись)
- Быстрая верификация

Шифрование:
- AES-256-GCM для шифрования приватных ключей
- scrypt для KDF (N=32768, r=8, p=1)
- Соль 32 байта для каждого ключа

Хэширование:
- SHA-256 для документов
- Хэш блока включает: высоту, prev_hash, document_hash, timestamp

Защита от атак
- Rate limiting (настраиваемый)
- Validation всех входных данных
- Проверка подписей перед сохранением
- HTTPS/TLS поддержка (конфигурация)
- Отзыв скомпрометированных ключей

Консенсус

Динамический расчёт

```
active_keys = GetActiveKeys(last 24h)
required = max(2, active_keys / 2 + 1)  # 51% + 1
```

Пример работы

```
Зарегистрировано ключей: 50
Активных за 24 часа:     10
Требуется для консенсуса: 10/2 + 1 = 6 подписей

Клиент 1 подписал (1/6)
Клиент 2 подписал (2/6)
...
Клиент 6 подписал (6/6) ✅ КОНСЕНСУС
```

Роль P2P в консенсусе

Благодаря P2P сети:
- Клиенты узнают о новых блоках быстрее
- Подписи распространяются между пирами
- Консенсус достигается за меньшее время
- Нет зависимости от сервера для получения блоков

Использование

Развёртывание

Docker (рекомендуется)

```
# Production стек
./scripts/deploy.sh --production

# Только сервер
./scripts/deploy.sh --server-only
```

Переменные окружения

```
export CHAINDOCS_AUTH_TOKEN="secure_token"
export CHAINDOCS_DB="/var/lib/chaindocs/blockchain.db"
export CHAINDOCS_CONFIG="/etc/chaindocs/config.json"
export CHAINDOCS_P2P_PORT="8081"
```

Примеры использования

1. Генерация ключа

```
./bin/keygen -password mypass -out key.enc
```

2. Регистрация ключа

```
curl -X POST http://localhost:8080/api/register \
  -d '{"public_key":"abc123..."}'
```

3. Загрузка с подписью

```
./scripts/sign-document.sh -k key.enc -p pass -f doc.pdf

curl -X POST http://localhost:8080/api/upload \
  -F "file=@doc.pdf" \
  -F "document_signature=..." \
  -F "public_key=abc123..." \
  -F "category=diplomas"
```

4. Bulk загрузка

```
curl -X POST http://localhost:8080/api/upload/bulk \
  -F "files=@d1.pdf" \
  -F "files=@d2.pdf" \
  -F "category=diplomas"
```

Преимущества для организаций

Для бизнеса

Неизменяемость:

- Документы не могут быть изменены post-factum
- Полная история операций
- Юридическая значимость

Снижение издержек:

- Нет необходимости в доверенных посредниках
- Автоматизация процессов подтверждения
- Снижение бумажного документооборота

Аудит:

- Полная трассируемость операций
- Быстрая проверка подлинности
- Соответствие регуляторным требованиям

Для IT отделов

Простота развёртывания:

- Docker контейнеризация
- Один бинарный файл
- Минимальные требования

Интеграция:

- REST API
- Swagger/OpenAPI документация
- Webhooks для уведомлений

Мониторинг:
- Prometheus метрики
- Grafana дашборды
- Health checks

Для безопасности

Криптография:
- Промышленные стандарты (Ed25519, AES-256)
- Аудит безопасности
- Отзыв ключей

Контроль доступа:
- Аутентификация в веб-интерфейсе
- Разграничение прав
- Логирование операций

Сравнение с аналогами

Обзор рынка систем документооборота

На рынке существует несколько категорий систем для управления документами:

1. Централизованные БД (PostgreSQL, MongoDB)
- Традиционный подход
- Требуют доверия к администратору
- Нет криптографической гарантии

2. Корпоративные системы (1С:Документооборот, Directum)
- Готовые решения для бизнеса
- Сертифицированная ЭЦП
- Дорогая лицензия и поддержка

3. Частные блокчейны (Hyperledger Fabric, Ethereum Private)
- Децентрализация
- Сложная настройка и поддержка
- Общего назначения, не для документов

4. Специализированные блокчейн-системы (ChainDocs)
- Децентрализация + простота
- Специализация на документах
- Бесплатно (Open Source)

ChainDocs vs 1С:Документооборот

1С:Документооборот — популярная система в РФ для управления документами.

[table] | Критерий | ChainDocs | 1С:Документооборот | | Цена лицензии | ✅ Бесплатно | ❌ От 150,000₽ | | Ежегодные платежи | ✅ Нет | ❌ 20-30% от стоимости | | Установка | ✅ Docker, 1 команда | ❌ Сложная, специалисты | | Требования к серверу | ✅ Минимальные | ❌ 1С:Сервер + СУБД | | Неизменяемость | ✅ Блокчейн гарантия | ⚠️ На уровне приложения | | Децентрализация | ✅ P2P сеть | ❌ Централизованная БД | | Отказоустойчивость | ✅ Нет

единой точки | ⚠️ Репликация 1С:Сервера | | Криптография | ✅ Ed25519, AES-256 | ⚠️ Стандартная | | Аудит | ✅ Встроенный в блокчейн | ⚠️ Нужно настраивать | | Интеграции | ✅ REST API, Swagger | ⚠️ 1С-специфичные | | Масштабирование | ✅ Горизонтальное (P2P) | ⚠️ Вертикальное | | Независимость | ✅ Go бинарник | ❌ Зависимость от 1С | | Юр. значимость | ⚠️ В разработке | ✅ Есть (сертифицировано) | | Мобильность | ⚠️ В разработке | ⚠️ Есть (1С:Мобайл) |

Вывод: ChainDocs лучше подходит для децентрализованных сценариев с гарантией неизменности. 1С лучше для готовой интеграции с бухгалтерией.

ChainDocs vs Directum

Directum — ещё одна популярная ECM-система.

[table] | Критерий | ChainDocs | Directum | | Цена | ✅ Бесплатно | ❌ От 200,000₽ | | Открытость | ✅ Open Source | ❌ Проприетарное | | Блокчейн | ✅ Есть | ❌ Нет | | P2P | ✅ Есть | ❌ Нет | | REST API | ✅ Полное | ⚠️ Ограниченное | | Docker | ✅ Есть | ❌ Нет |

ChainDocs vs Hyperledger Fabric

Hyperledger Fabric — платформа для частных блокчейнов.

[table] | Критерий | ChainDocs | Hyperledger Fabric | | Простота | ✅ 1 команда | ❌ Сложно (K8s) | | Специализация | ✅ Документы | ❌ Общего назначения | | Консенсус | ✅ 51% динамический | ❌ Фиксированный | | Сообщество | ⚠️ Малое | ✅ Большое | | Поддержка | ⚠️ Community | ✅ Enterprise |

ChainDocs vs PostgreSQL

[table] | Критерий | ChainDocs | PostgreSQL | | Неизменяемость | ✅ Гарантия | ⚠️ Приложение | | Децентрализация | ✅ Полная | ❌ Нет | | Аудит | ✅ Встроенный | ⚠️ Настройка | | Подписи | ✅ Крипто | ❌ Нет | | P2P | ✅ Есть | ❌ Нет |

Когда выбирать ChainDocs

✅ Выбирайте ChainDocs если:

- **Нужна гарантия неизменности документов** — блокчейн обеспечивает криптографическую защиту
- **Требуется децентрализация** — P2P сеть без единой точки отказа
- **Ограниченный бюджет** — бесплатно, нет лицензионных платежей
- **Нужна простая интеграция** — REST API, Swagger документация
- **Критична отказоустойчивость** — P2P репликация данных
- **Нет инфраструктуры 1С** — не нужно покупать 1С:Сервер
- **Нужен полный аудит** — встроенная трассируемость операций
- **Планируется масштабирование** — горизонтальное через P2P

Когда выбирать аналоги

⚠️ Выбирайте 1С:Документооборот если:

- **Уже используется 1С:Предприятие** — готовая интеграция с бухгалтерией
- **Требуется сертифицированная ЭЦП** — пока ChainDocs в разработке
- **Нужна готовая интеграция с госуслугами** — 1С сертифицирован

- **Есть бюджет на лицензию** — от 150,000₽ + поддержка
- **Нужна официальная поддержка вендора** — 1С предоставляет

⚠️ Выбирайте Hyperledger если:

- **Нужен кастомный блокчейн** — Fabric гибче
- **Есть команда блокчейн-разработчиков** — сложно в настройке
- **Требуется enterprise поддержка** — IBM, Intel поддерживают

⚠️ Выбирайте PostgreSQL если:

- **Не нужна неизменяемость** — документы могут изменяться
- **Доверенная среда** — нет риска внутренних атак
- **Нужна максимальная производительность** — бд быстрее блокчейна

Итоговая матрица выбора

```
┌──────────────────────────────────────────────────────────┐
│                      Выбор системы                       │
├──────────────────────────────────────────────────────────┤
│ Нужна гарантия неизменности? → ChainDocs ✅              │
│ Уже есть 1С:Предприятие? → 1С:ДО ⚠                       │
│ Ограниченный бюджет? → ChainDocs ✅                      │
│ Нужна сертифицированная ЭЦП? → 1С:ДО ⚠                  │
│ Критична отказоустойчивость? → ChainDocs ✅              │
│ Нужна интеграция с бухгалтерией? → 1С:ДО ⚠               │
│ Нужно горизонтальное масштабирование? → ChainDocs ✅     │
│ Есть команда блокчейн-разработчиков? → Hyperledger ⚠     │
└──────────────────────────────────────────────────────────┘
```

Недостатки и ограничения

Текущие ограничения

- Производительность ограничена bbolt (1000 операций/сек)
- Требуется доверие к серверу для веб-интерфейса
- Нет шардинга (планируется)

Планы развития

- [ ] Интеграция с LDAP/AD для аутентификации
- [ ] Мобильное приложение для подписи
- [ ] Публичный реестр документов
- [ ] Юридическая значимость (ЭЦП)
- [ ] Мульти-регион развёртывание

Заключение

ChainDocs — это современная децентрализованная система документооборота, которая решает критические проблемы современного бизнеса:

- **Неизменяемость** через блокчейн
- **Децентрализация** через P2P сеть
- **Безопасность** через криптографию
- **Простота** через продуманный API

Система готова к production развёртыванию и может быть использована организациями любого размера.

Контакты

- GitHub: https://github.com/EvgeniiAndronov/ChainDocs
- Документация: docs/chaindocs.pdf
- Версия: 1.2.0
- Статус: ✅ Production Ready